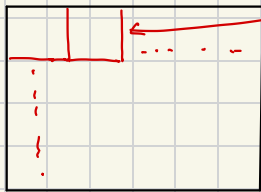


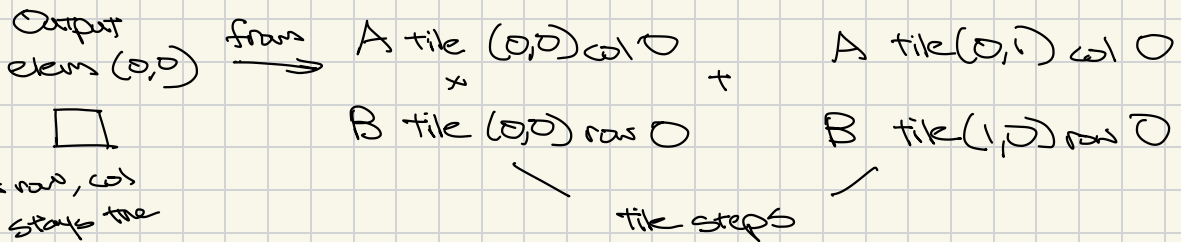
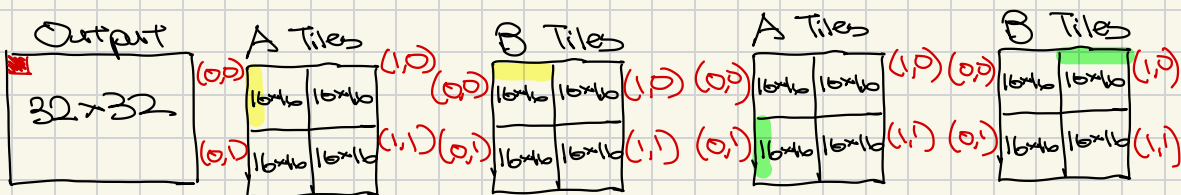
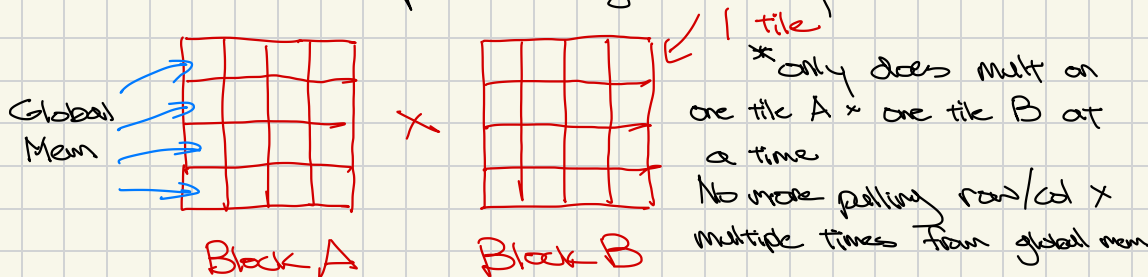
Tiled Matrix Mult Design

- Break down problem into smaller pieces



- Store data in shared memory

- Prevents redundant pulls from global memory in same block



* row, col stays the same

Move along y for col tiles
x for row tiles

Given $t_s =$ tile size along 1 dim
do shared dimension

For output $(j, k) = \sum_{i=0}^{t_s-1} \text{tile}(j/t_s, i) \text{col } k/t_s \cdot \text{tile}(i, k/t_s) \text{row } j/t_s$

Go along entire column in A

Go along entire row in B

So instead of $\begin{bmatrix} 0 & \dots & n \end{bmatrix} \times \begin{bmatrix} 0 \\ \vdots \\ m \end{bmatrix}$ all pulled at the same time

Split up into chunks of row and column of size t_s
 ↳ That way we can make in shared memory for quick access

Declare shared memory: __shared__

Dimensions: Given $\underbrace{[n \times k]}_{\text{Matrix A}} \times \underbrace{[k \times m]}_{\text{Matrix B}} = \underbrace{[n \times m]}_{\text{Matrix C}}$

- Tile Size of 1 Dim (t_s)

- $t_s \times t_s$ should be multiple of 32 (warp size)

- $2 \times t_s \times t_s \times 4$ ↳ shared mem size

Constraints (2 matrices) (num elems per tile) (4 bytes per float)

- Smaller tile size

- Since one block has two tiles at once

- Smaller $t_s \rightarrow$ more blocks per sm

↳ More options for switching

↳ Better gpu utilization / lower latency
 better throughput

- Bigger tile size

- More reuse per block

- Don't have to load as much from memory

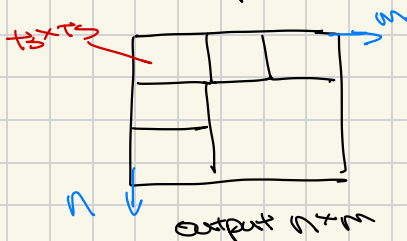
- Tile can also be rectangular

- One thread per output elements

- Pick thread per block & block size accordingly

Block Dim - 2D map of threads in tile $[t_s][t_s]$

Grid Dim - 2D map of tile in matrix



Grid Dim $[(n+t_s-1)/t_s][(m+t_s-1)/t_s]$

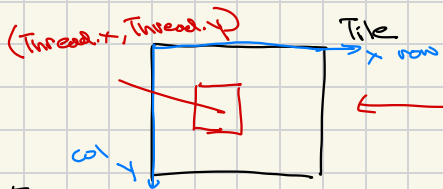
of tile steps

$= (k+t_s-1)/t_s$

↳ shared dimension

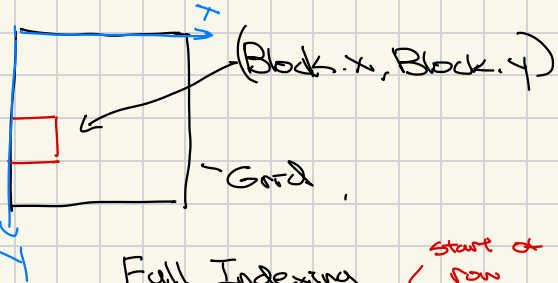
Tiled Kernel Design

Output Elem Index



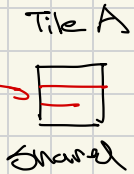
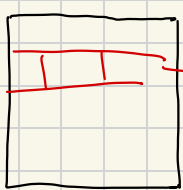
Find index in tile
 $\text{Thread.y} * \text{num_cols} + \text{Thread.x}$
 to go down the full row in grid

Start of block in matrix
 $= \text{Block.y} * \text{ts} * \text{num_cols} + \text{Block.x} * \text{ts}$
 row of grid col



Full Indexing
 $(\text{Block.y} * \text{ts} + \text{Thread.y}) * \text{num_cols} + \text{Block.x} * \text{ts} + \text{Thread.x}$
 start of row col index

Pulling Global into Shared



$t = \text{tile step}$

A row: $\text{Block.y} * \text{ts} + \text{Thread.y}$
 A col: $\text{ts} * t + \text{Thread.x}$
 slides across entire row

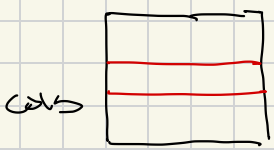


B row: $\text{ts} * t + \text{Thread.y}$
 B col: $\text{Block.x} * \text{ts} + \text{Thread.x}$
 slides across entire col

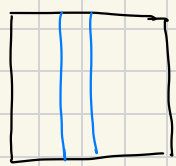
Synchronize to make sure shared mem is fully updated

for Product from $i=0$ to ts

Start Idx
 $\text{row} = \text{Thread.y}$
 $\text{col} = 0$



x



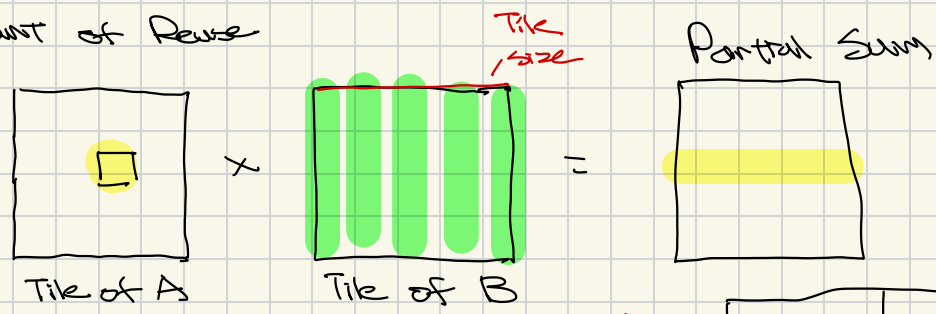
Start Idx
 $\text{row} = 0$
 $\text{col} = \text{ThreadIdx.x}$
 Step
 $+ i$ in row every step

Step

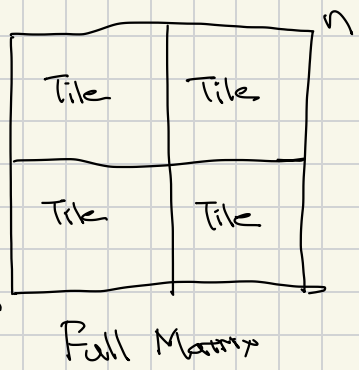
$+ i$ in col every step

* Synchron again to load new data

Amount of Reuse



Reused tile size times



for output $m \times n$ matrix
each element (x, y)
depends on

$A(x, 0 \dots k)$ and $B(0 \dots m, k)$

if there are $m \times n$ output elements

we use $m \times n (A(x, 0 \dots k))$ and $m \times n (B(0 \dots m, k))$

but since we have tiling we reuse one element
{Tile size} times per tile

so the # of fetches from global memory is now

$$\frac{m \times n \times k}{\text{tile size}} + \frac{m \times n \times k}{\text{tile size}} = \frac{2 \times m \times n \times k}{\text{tile size}}$$

↑
fetches for
A
↑
fetches for
B